



Conditional Memory via Scalable Lookup:
A New Axis of Sparsity for Large Language Models

Engram

论文解读

东川路第一可爱猫猫虫

主要内容

- LLM用计算模拟检索
- N -gram
- Engram的流程

分词器压缩 多头哈希 上下文感知门控

- Engram的优化

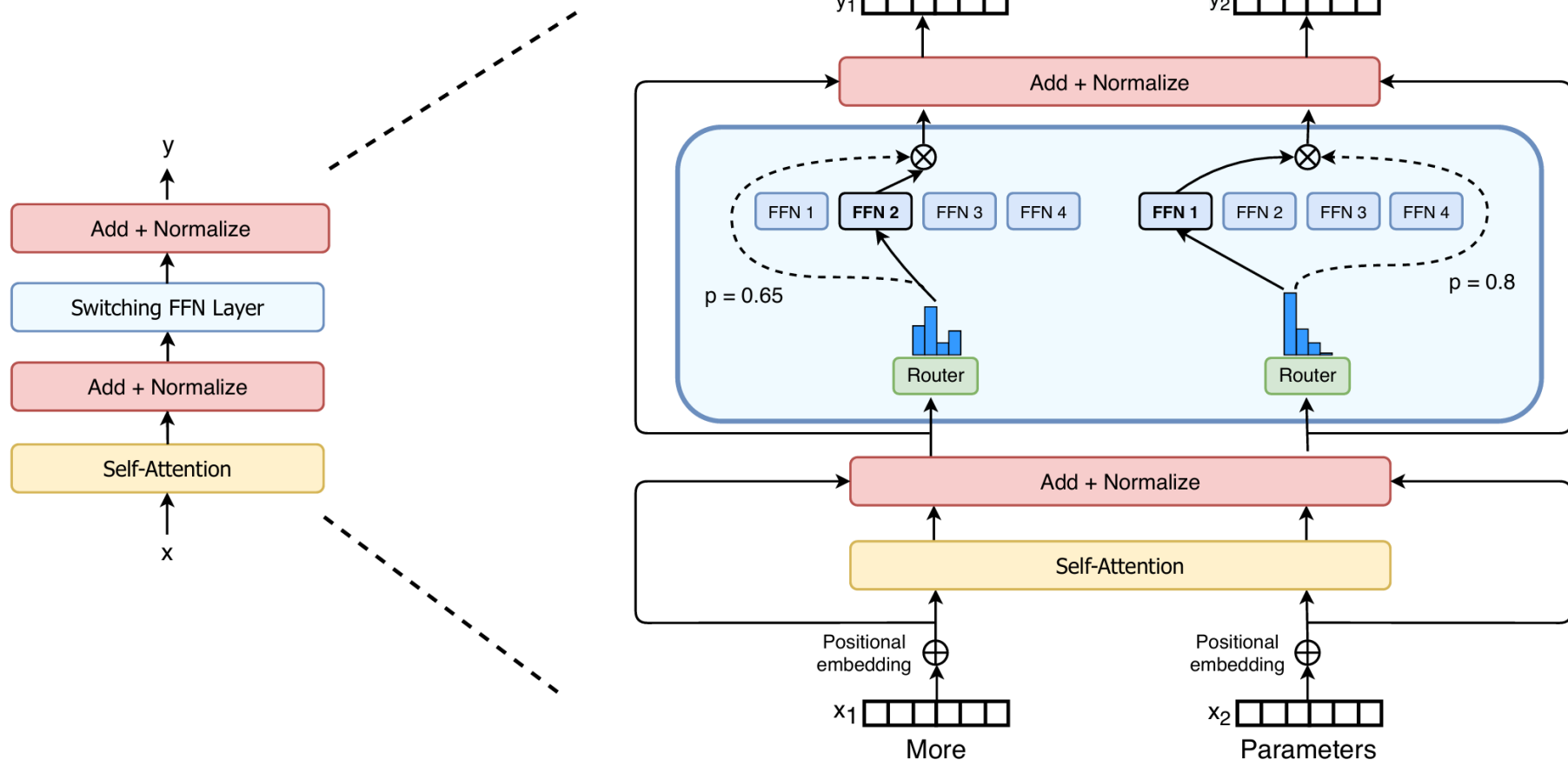
与mHC适配 MoE解耦计算与记忆 多级缓存

- Engram的实验

槽位越多越好吗 参数如何分配 放置在哪一层最好

MOE: 条件计算

While Mixture-of-Experts (MoE) scales capacity via conditional computation, Transformers lack a native primitive for knowledge lookup, forcing them to inefficiently simulate retrieval through computation. To address this, we introduce conditional memory as a complementary sparsity



LLM如何理解“Diana, Princess of Wales”

- 一层一层看LLM对这个词组的理解过程
- 用PatchScope把LLM对词在每一层的理解翻译成自然语言

1-2 : Country in the United Kingdom
3 : Country in Europe
4 : Title held by female sovereigns in their own right or by queens consort
5 : Title given to the wife of the Prince of Wales (and later King)
6 : Diana, Princess of Wales (1961-1997), the first wife of Prince Charles, Prince of Wales, who was famous for her beauty and humanitarian work

LLM: simulate retrieval through computation

- LLM通过多层的注意力和前馈网络
才成功解析了一个多token的词
- 本质上是用计算来模拟检索
- Diana, Princess of Wales
是一个常见的静态知识
我们却用了六层网络来识别
宝贵的序列深度被浪费在了这种无意义的计算上
- 明明可以静态查表
大模型却进行了昂贵的运行时重建

While Mixture-of-Experts (MoE) scales capacity via conditional computation, Transformers lack a native primitive for knowledge lookup, forcing them to inefficiently simulate retrieval through computation. To address this, we introduce conditional memory as a complementary sparsity axis, instantiated via Engram, a module that modernizes classic N -gram embedding for $O(1)$ lookup. By formulating the Sparsity Allocation problem, we uncover a U-shaped scaling law that optimizes the trade-off between neural computation (MoE) and static memory (Engram).

- N -gram

文本中由 N 个连续单词（或Token）组成的序列

Unigram Bigram Trigram

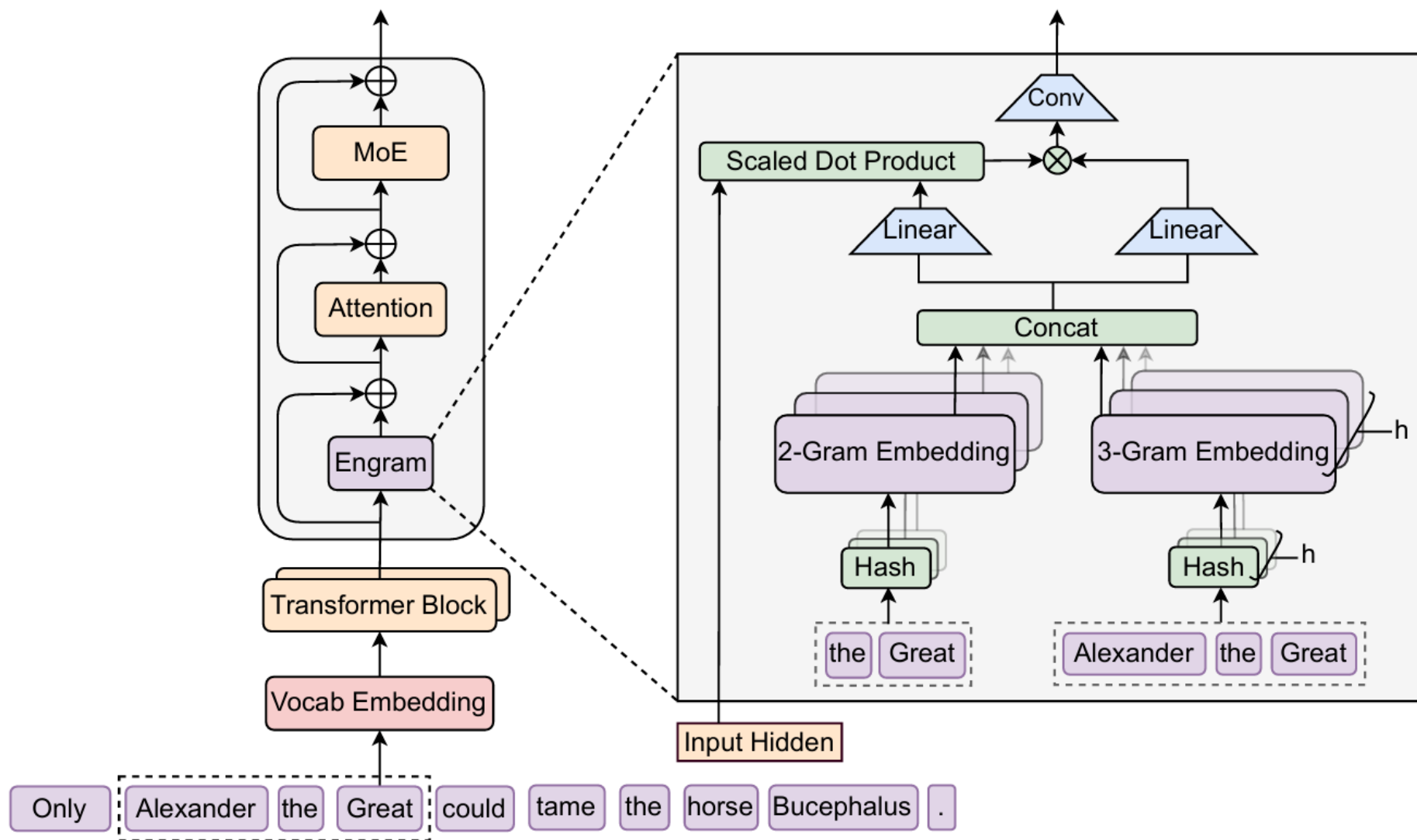
- 早期：基于 n -gram的语言模型
- 本文： N -gram embedding

把像“Diana, Princess of Wales”这样的 N -gram 存储起来以供查找

为什么重提 N -gram

- MoE的成功在于稀疏性
条件计算
- 语言建模包含两个性质截然不同的子任务
组合推理和知识检索
- 推理任务需要计算
- 知识检索需要记忆
命名实体、固定搭配、公式化词组是静态的
- Engram 条件记忆
 N -gram的升级版

Engram究竟是怎么做的



传统的无损重建

- Embedding词嵌入

把词语嵌入为高维向量

- lossless reconstruction 无损重建

Apple

apple

_apple

- 这样的分词算法

对大小写、空格、特殊字符非常敏感

造成了词表浪费

Tokenizer Compression 分词器压缩

- Engram引入了一个词表投影层
- 用一个满射函数

把所有tokenID折叠为规范标识符

$$x'_t = \mathcal{P}(x_t)$$

有效的词表大小减少了23%

- 构建N-gram

对于当前位置t, Engram 会取过去n个位置的**压缩后**Token

$$g_{t,n} = (x'_{t-n+1}, \dots, x'_t)$$

$g_{t,n}$ 就是我们后续用来检索的key

Multi-Head Hashing 多头哈希

- 出现问题：直接参数化所有可能的 N-gram 组合空间是不可行的

- 解决方法：哈希映射

我们买不起一亿个槽位，就买一百万个槽位

无论有多少词组，都塞进这仅有的一百万个槽位里

- 出现问题：哈希冲突

由于记忆槽位有限，不同语义的词组会被分入同一个槽位

- 解决方法：多头哈希

为每个 N-gram 阶数 n 使用 K 个不同的哈希头

单点可能出错，但多点难以同时出错

Multi-Head Hashing 多头哈希

- 用K个不同的哈希函数并行算K次

$$z_{t,n,k} \triangleq \varphi_{n,k}(g_{t,n}), \quad e_{t,n,k} = E_{n,k}[z_{t,n,k}]$$

Multiplicative-XOR Hash

每个嵌入表 $E_{n,k}$ 的大小 $M_{n,k}$ 被设定为素数

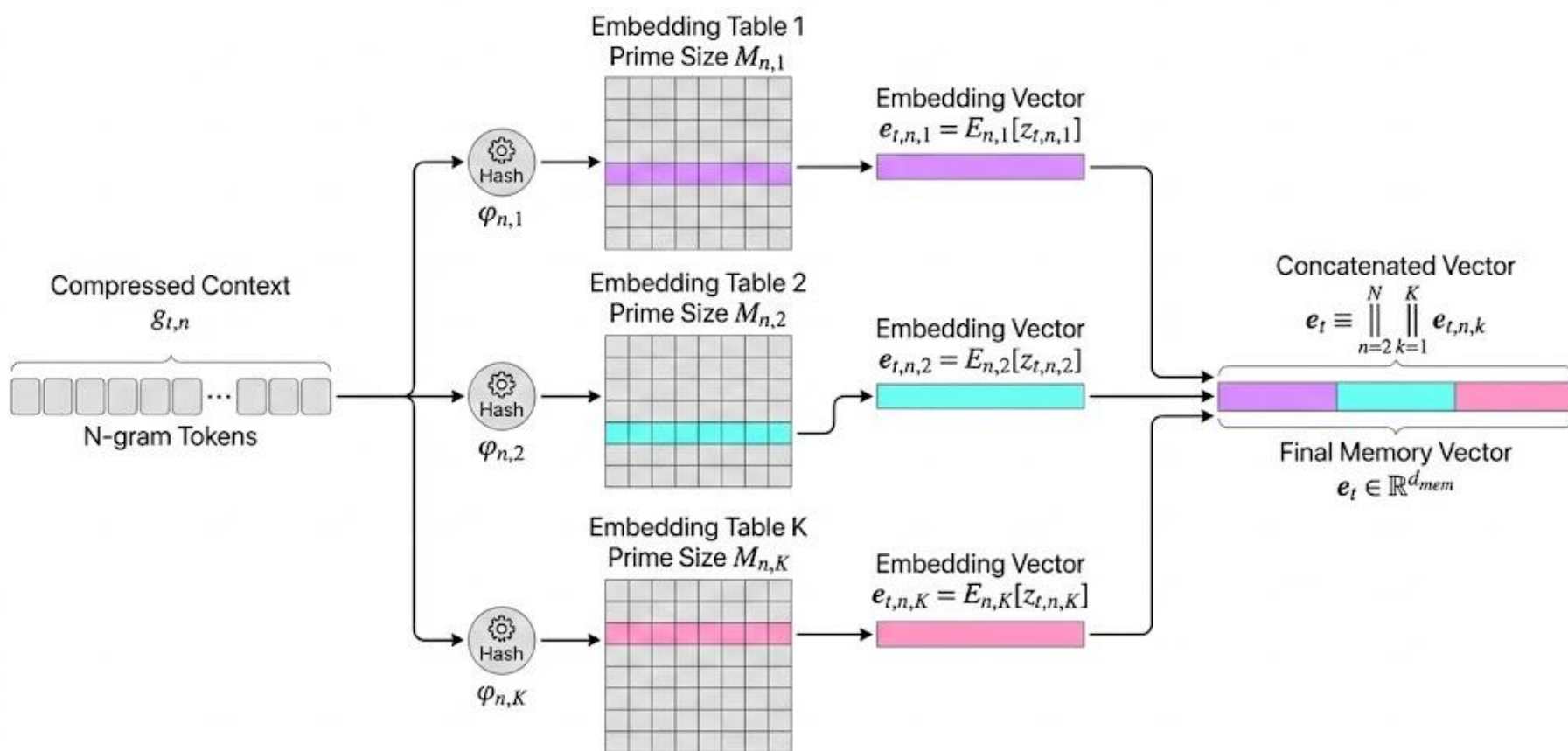
- 全拼接

$$e_t \triangleq \left\|_{n=2}^N \left\|_{k=1}^K e_{t,n,k} \right\|$$

就得到了t时刻的信息丰富的记忆向量 e_t

Multi-Head Hashing 多头哈希

Multi-Head Hashing Retrieval



$$z_{t,n,k} \triangleq \varphi_{n,k}(g_{t,n})$$

$$\mathbf{e}_{t,n,k} = \mathbf{E}_{n,k}[z_{t,n,k}]$$

$$\mathbf{e}_t \triangleq \big\|_{n=2}^N \big\|_{k=1}^K \mathbf{e}_{t,n,k}$$

Context-aware Gating 上下文感知门控

- 现在我们得到了拼接好的记忆向量 e_t
 e_t 是从静态知识库查表得到的
由于哈希冲突或者多义词的存在, e_t 可能含有错误的含义
- e.g. Apple released a new phone
“水果苹果” “科技公司苹果” “? ? ? ”
- 解决办法:
用主干网络的当前状态动态筛选 e_t 里的信息
只允许合适的信息流入主干网络

e_t : 不含上下文的记忆向量

h_t : 聚合了上下文的当前隐藏状态

- h_t 作为 Query, 用 e_t 得到 Key 和 Value

$$k_t = W_K e_t, \quad v_t = W_V e_t$$

- 计算 Query 和 Key 的相似度

$$\alpha_t = \sigma \left(\frac{\text{RMSNorm}(h_t)^\top \text{RMSNorm}(k_t)}{\sqrt{d}} \right)$$

$\alpha_t \rightarrow 1$ 说明记忆向量和当前上下文语境**吻合**

$\alpha_t \rightarrow 0$ 说明记忆向量和当前上下文语境**冲突**

- $\tilde{v}_t = \alpha_t \cdot v_t$

用分数 α_t 对记忆的 Value v_t 进行过滤得到 $\tilde{v}_t$

- $Y = \text{SiLU} \left(\text{Conv1D} \left(\text{RMSNorm}(\tilde{V}) \right) \right) + \tilde{V}$

Engram与mHC的适配

- Manifold-Constrained Hyper-Connections, mHC
- 如果每个残差流之间 Engram完全各自独立
 参数量和计算量过大
- 共享部分：
 稀疏嵌入表 e_t
 Value 投影矩阵 W_V
- 独立部分：
 Key 投影矩阵 $W_K^{(m)}$

时间管理大师：Engram解耦计算与记忆

- MoE的路由依赖于上一层的计算
不能预先加载专家，因为不知道会用到谁
- Engram的查表索引仅仅取决于输入Token序列
这种确定性与MoE不同
索引在计算开始前就是已知的
- 异步预取
GPU计算前面层的时候，CPU就可以传输Engram所需数据
- 掩盖延迟
前面层的计算作为缓冲区，防止GPU停顿

放置位置的Trade-off

- 系统角度

 - 希望把 Engram 放得**深一点**

 - 前面的层越多，GPU 计算的时间就越长，留给 CPU 搬运数据的时间窗口就越充裕

- 模型角度

 - 希望把 Engram 放得**浅一点**

 - 越早注入知识，越能替模型省下死记硬背的力气，效果越好

- 模型性能和系统延迟的权衡下

 - 确定最合适的放置位置

多级缓存

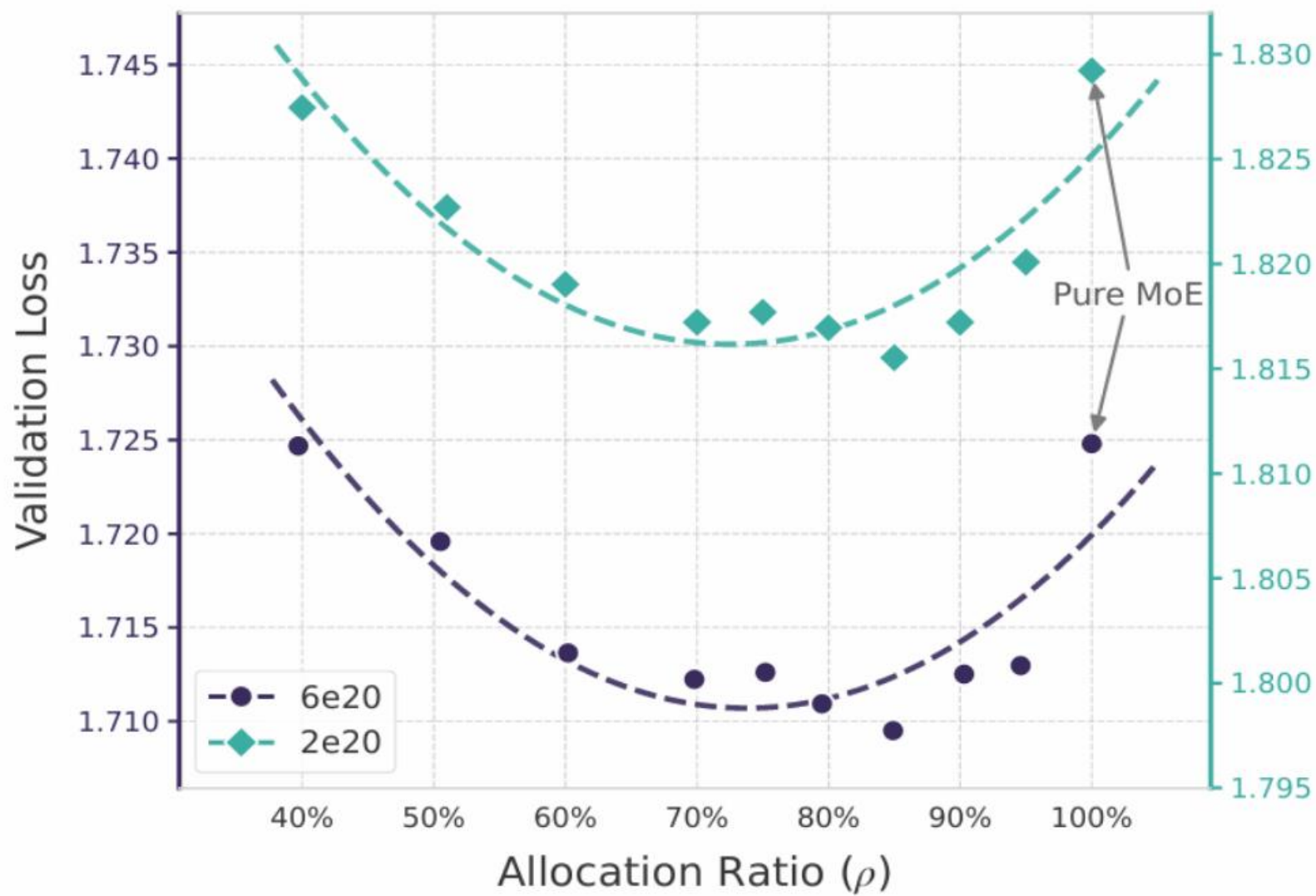
- 自然语言N-grams本质上遵循Zipfian分布
20%的词汇出现了80%的次数
- 更快的存储层
GPU HBM 或 Host DRAM
放置使用最频繁的词
- 较慢的高容量介质
NVMe SSD
放长尾的罕见词
- Engram扩展到巨大的内存容量，而对有效延迟的影响最小

模型参数该如何在MoE和Engram之间分配

- P_{tot} : 所有的可训练参数
- P_{act} : 激活的参数
- $P_{sparse} \triangleq P_{tot} - P_{act}$: 未激活的参数
 - 免费午餐
 - 增加模型容量但并不增加计算成本
- P_{sparse} 该如何分配给MoE和Engram
- $\rho = \frac{P_{sparse}^{MoE}}{P_{sparse}}$

最优分配比率

ρ 约为75%到80%时为最优

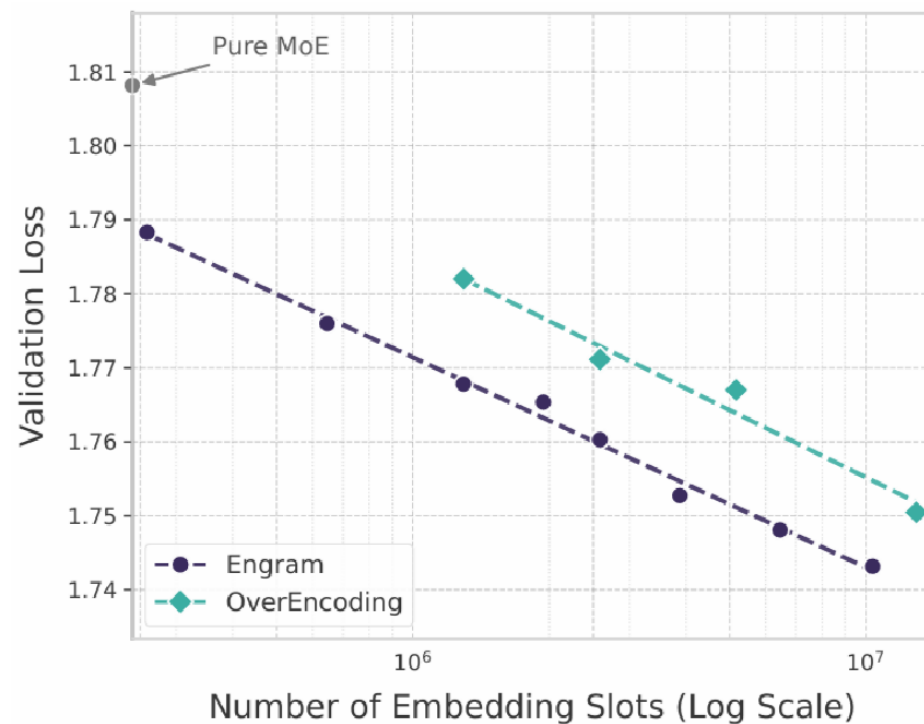


无限内存机制下的 Engram

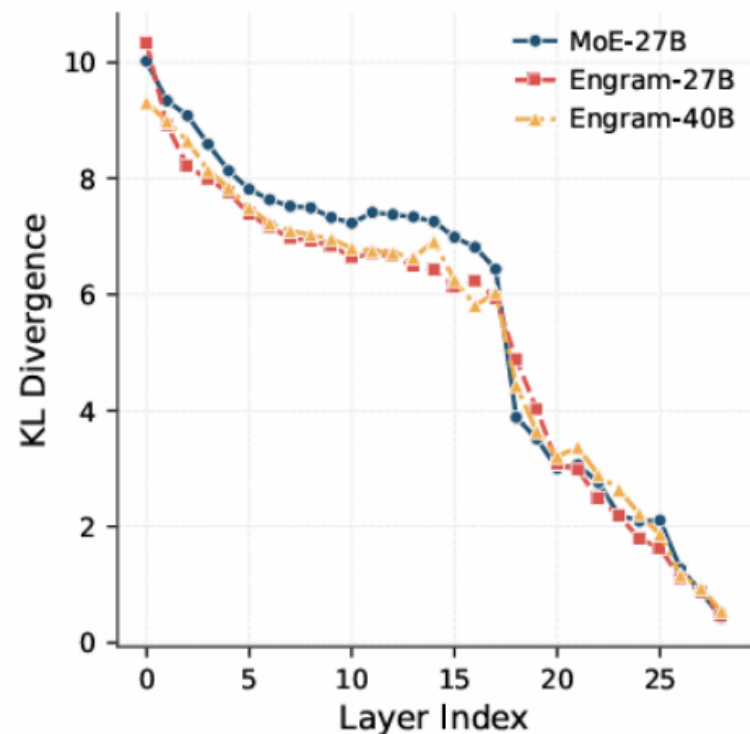
- 计算量不变，模型结构不变

增加Engram的记忆槽位，效果会怎么变？

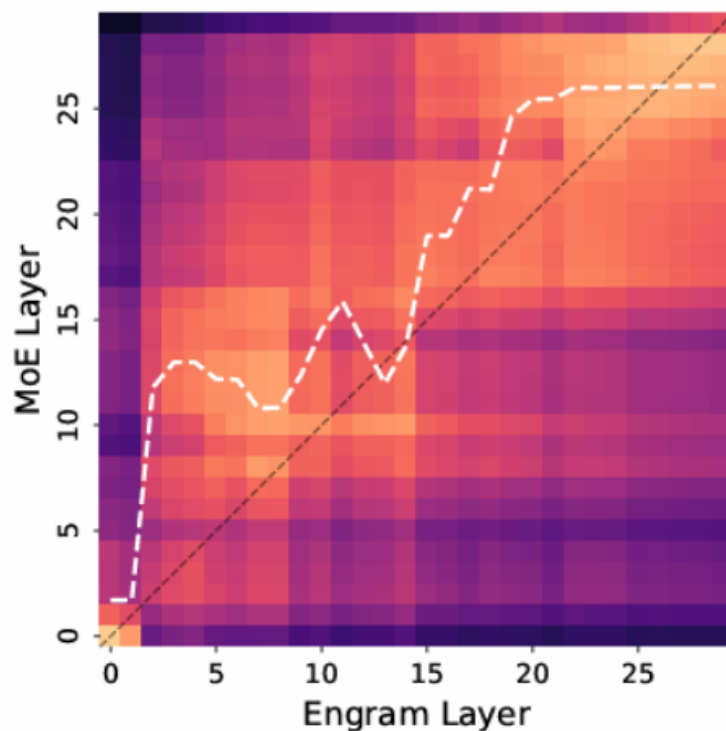
实验范围：从 25.8 万个槽位一直加到 1000 万个槽位



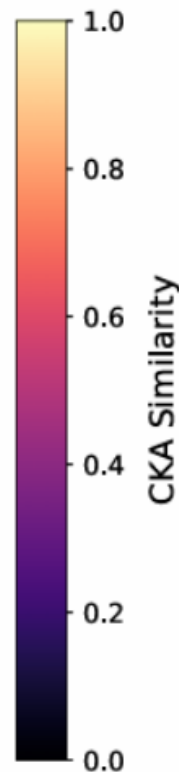
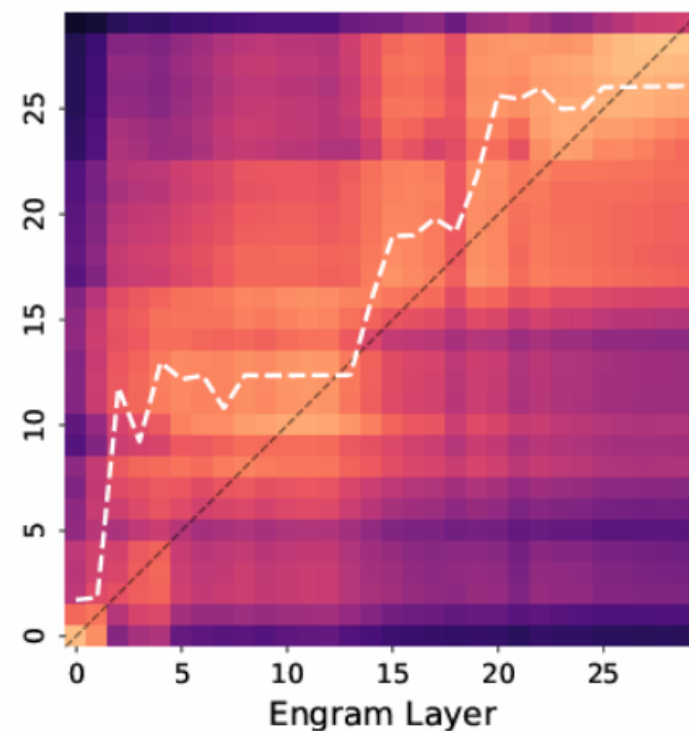
(a) Layer-wise KL Divergence by LogitLens



(b) CKA map: Engram-27B vs MoE-27B



(c) CKA map: Engram-40B vs MoE-27B



- Engram模型的KL散度下降得比MoE快得多，特别是在前几层
- Centered Kernel Alignment衡量两个神经网络层的表示有多像
原本MoE模型需要很多层计算得到的知识
Engram模型在浅层就已得到

Engram放在哪一层

